

Message Authentication and Hash Functions

3.1 Message Authentication

3.2 Hash Functions

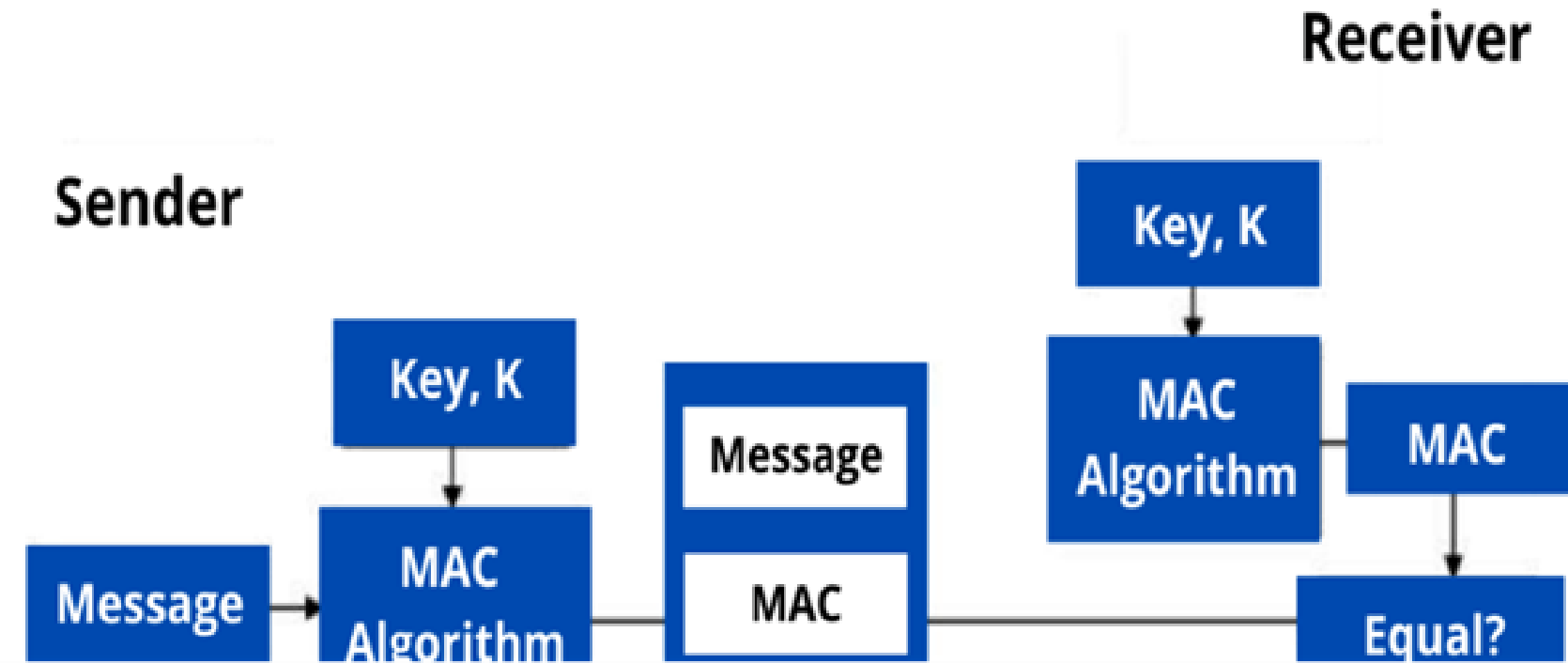
3.3 Message Digests: MD4 and MD5

3.4 Secure Hash Algorithms: SHA-1

3.5 HMAC

3.6 Digital Signature

Message authentication



- Message authentication is the process of **verifying the integrity of a message to ensure that it has not been altered or tampered with during transmission**. This is achieved by adding a **digital signature or a message authentication code (MAC)** to the message, which can be used to **verify its authenticity**.
- One common method of message authentication is through the use of **symmetric-key cryptography**. In this approach, the **sender** and receiver **share a secret key** that is used to both encrypt and authenticate the message. The **receiver** can use the **same key to decrypt** the message and verify its authenticity.
- For example, suppose that **Alice wants to send a message to Bob**, and they have agreed on a shared secret key beforehand. **Alice** can use the **shared key** to generate a MAC for the message **by using a MAC Algorithm** and **adding the key** as a parameter. **Bob** can then **receive the message** and compute the MAC using the **same key and MAC Algorithm**. If the computed **MAC matches** the one sent by **Alice**, **Bob** can be sure that the **message has not been altered** during transmission and was indeed sent by **Alice**.

Message authentication

- There are several commonly used **message authentication codes** (MACs), including HMAC, which uses a cryptographic **hash function** in combination with a secret key to generate the MAC. Another common MAC is CBC-MAC, which uses a block cipher in CBC (**cipher-block chaining**) mode to generate the MAC.
- Message authentication is important for **ensuring the security and integrity** of electronic communications, particularly in situations where **sensitive or confidential** information is **being transmitted**. It is used in a wide range of applications, including **secure network communication protocols, digital signatures, and password storage**.

Message Authentication Code (MAC)

- A Message Authentication Code (MAC) is a cryptographic technique that is used to authenticate the **integrity and authenticity** of a message.
- It is a type of checksum that is generated by using a **secret key** and a cryptographic **MAC function** to compute a **fixed-size code** that is **appended to the message**. The **recipient** can then use the **same secret key and MAC function** to verify the MAC and ensure that the message has not been tampered with or modified during transmission.

The following are the general steps involved in generating and verifying a MAC:

1. A **secret key** is shared between the sender and the recipient.
2. The sender computes the MAC by applying a **cryptographic MAC function** to the message and the secret key.
3. The sender sends **the message along with the MAC** to the recipient.
4. The recipient uses the **same secret key** and **MAC function** to compute the MAC for the received message.
5. The recipient compares the computed MAC with the received MAC to verify the **integrity and authenticity** of the message.

Limitation

Key distribution:

MACs require a shared secret key between the sender and receiver. This means that the key needs to be distributed securely, which can be **challenging in certain scenarios**.

Limited scope:

MACs only provide integrity and authenticity protection for a single message. They do not protect against other types of attacks, such as **replay attacks or man-in-the-middle** attacks.

Limited verification:

MACs only verify the integrity of the message and the authenticity of the sender, but they do not provide any **information about the content of the message itself**.

Replay attacks:

If a MAC tag is intercepted by an attacker, they can **use it to replay** the message to the recipient, which could compromise the integrity of the system.

Key reuse:

If the **same key is used to generate multiple MACs**, an attacker who can obtain two or more MACs can potentially derive the key and compromise the security of the system.

Hash function

- A hash function is a **mathematical algorithm** that takes in an input (also known as the "message") and produces a **fixed-length output**, known as a **hash or message digest**. The main purpose of a hash function is to ensure the **integrity of data** by verifying that it has not been **altered or corrupted** during transmission or storage.

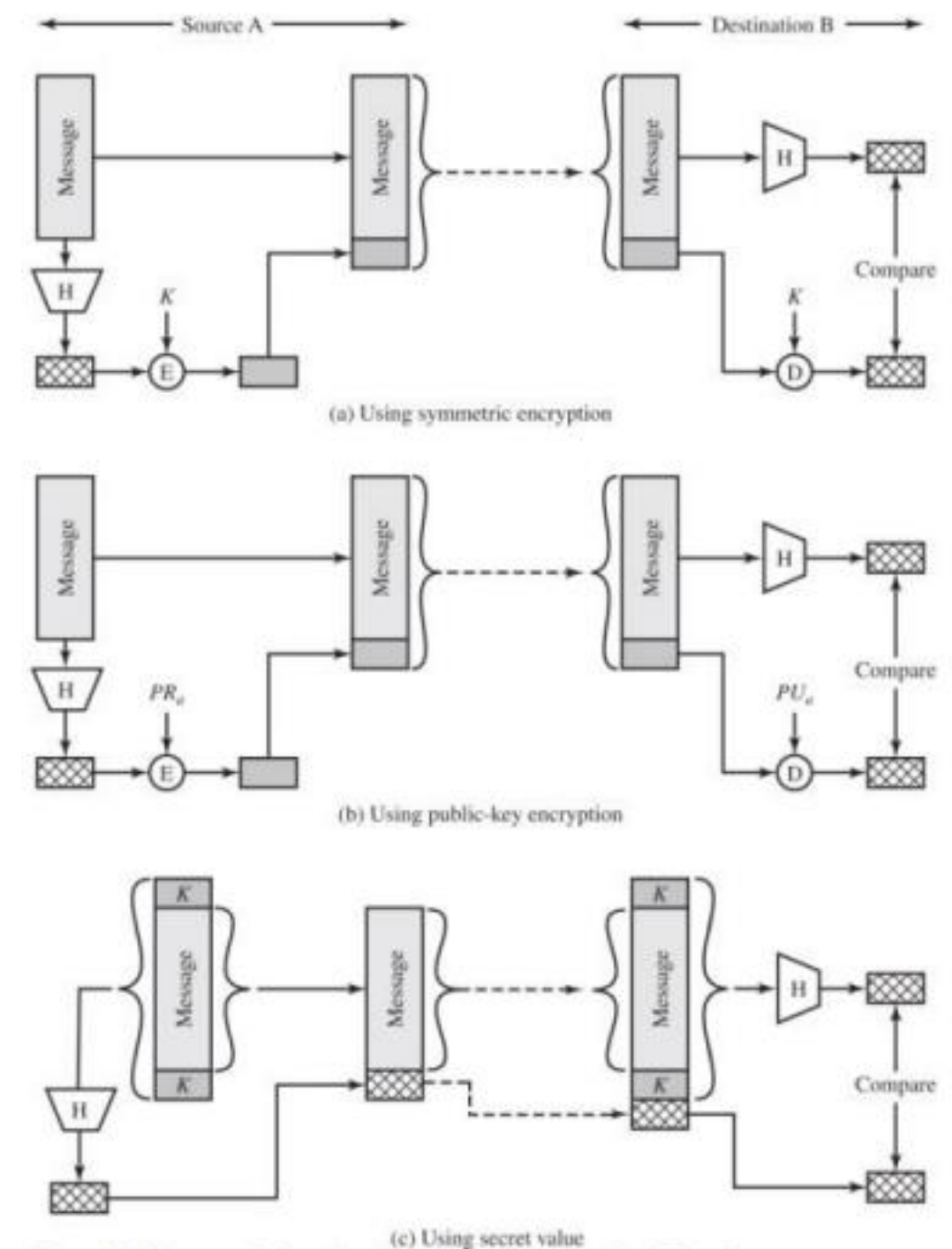
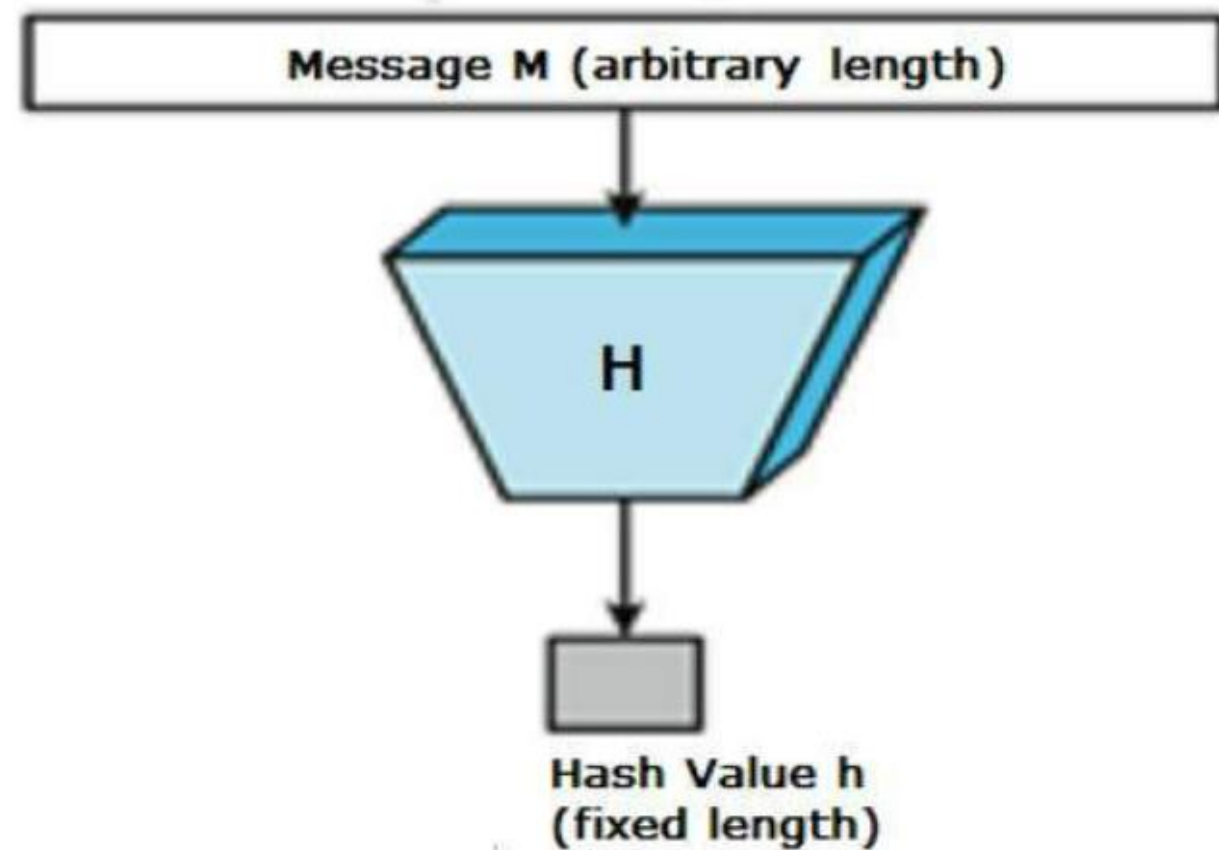


Figure 2.5 Message Authentication Using a One-Way Hash Function

Hash function Features

- **Deterministic:** Given the same input, the hash function will always produce the same output.
- **Fast:** A good hash function should be able to produce a hash value quickly, even for large amounts of data.
- **Uniformity:** A hash function should distribute its output uniformly across its range, meaning that every possible output value should have an equal probability of being produced.
- **Non-invertible:** A hash function should be difficult, if not impossible, to reverse. Given the hash value, it should be extremely difficult to determine the original input message.
- **Collision resistance:** A hash function should produce different hash values for different input messages. It should be computationally infeasible to find two different input messages that produce the same hash value.
- **Fixed length output:** A hash function should produce a fixed length output regardless of the size of the input message.
- **Sensitivity to input:** A small change in the input message should produce a significant change in the hash value.

Message-Digest: MD4

- MD4 (Message-Digest algorithm 4) is a cryptographic **hash function** developed by **Ronald Rivest** in 1990. It is designed to **produce a 128-bit hash value** from input messages of **arbitrary length**.

Features:

1. **Fast and efficient:** MD4 is designed to be fast and efficient, which makes it useful in situations where **large amounts of data need to be processed quickly**.
2. **Variable length input:** MD4 can process **input messages of variable length** and produce a **fixed-size output**.
3. **Widely used:** MD4 has been widely used in a variety of applications, including **data integrity checks, password hashing, and digital signatures**.

Applications:

1. **Data integrity checks:** MD4 can be used to verify the integrity of data by comparing the hash value of the original data with the hash value of the received data.
2. **Password hashing:** MD4 can be used to hash passwords for storage and verification. However, due to its vulnerabilities, stronger algorithms such as bcrypt or scrypt are now recommended.
3. **Digital signatures:** MD4 can be used to generate digital signatures to verify the authenticity of messages.

Message-Digest: MD4

Limitation

- **Vulnerable to attacks:** MD4 is now considered to be insecure due to its vulnerabilities to **collision attacks**.
- **Not suitable for new applications:** Due to its vulnerabilities, MD4 is no longer recommended for new applications. More secure hash functions such as **SHA-2** and **SHA-3** are no. recommended.
- **One-way function:** As a message digest algorithm, it is designed to be a one-way function, meaning that it is practically impossible to retrieve the original message from the hash value.

Message-Digest: MD5

MD5 (Message-Digest algorithm 5) is a widely-used cryptographic hash function that was designed by Ronald Rivest in 1991. It is designed to produce **a 128-bit hash value from input messages of arbitrary length.**

Features:

- **Fast and efficient:** MD5 is designed to be fast and efficient, which makes it useful in situations where large amounts of data need to be processed quickly.
- **Variable length input:** MD5 can process input messages of variable length and produce a fixed-size output.
- **Widely used:** MD5 has been widely used in a variety of applications, including data integrity checks, password hashing, and digital signatures.

Applications:

- **Data integrity checks:** MD5 can be used to verify the integrity of data by comparing the hash value of the original data with the hash value of the received data.
- **Password hashing:** MD5 can be used to hash passwords for storage and verification. However, due to its vulnerabilities, stronger algorithms such as bcrypt or scrypt are now recommended.
- **Digital signatures:** MD5 can be used to generate digital signatures to verify the authenticity of messages.

Message-Digest: MD5

Limitations:

- **Vulnerable to attacks:** MD5 has been found to be vulnerable to some attacks such as length extension attacks, making it less secure for some applications.
- **Not suitable for new applications:** Due to its vulnerabilities, MD5 is no longer recommended for new applications. More secure hash functions such as SHA-2 and SHA-3 are now recommended.
- **One-way function:** As a message digest algorithm, it is designed to be a one-way function, meaning that it is practically impossible to retrieve the original message from the hash value.
- Overall, while MD5 is still widely used in some legacy systems, it is generally considered less secure than newer hash functions and is not recommended for use in new applications.

Difference between MD4 and MD5:

- **Security:** MD4 is now considered to be insecure due to its vulnerabilities to collision attacks, whereas MD5 is also considered less secure than newer hash functions due to its vulnerabilities to length extension attacks.
- **Output length:** MD4 produces a 128-bit hash value, while MD5 also produces a 128-bit hash value.
- **Speed:** MD4 is faster than MD5, but this speed advantage comes at the cost of security.
- **Applications:** Both MD4 and MD5 can be used for data integrity checks, password hashing, and digital signatures, but MD5 is more widely used and supported in practice.
- **Recommendations:** Due to their known vulnerabilities, neither MD4 nor MD5 are recommended for use in new applications. More secure hash functions such as SHA-2 and SHA-3 are now recommended.

SHA-1 (Secure Hash Algorithm 1)

- **SHA-1** (Secure Hash Algorithm 1) is a cryptographic hash function designed by the National Security Agency (NSA) in 1995.
- It is designed to **produce a 160-bit hash value** from input messages of arbitrary length.

Procedures:

1. The SHA-1 algorithm operates on **512-bit message blocks**, and the input message is padded to the nearest multiple of 512 bits.
2. The message is then processed in **80 rounds of operations** that include **bitwise operations, logical functions, and modular arithmetic**.
3. The output of each round is combined with the output of the **previous round to generate the final hash value**.

Application:

- SHA-1 is commonly used for **data integrity checks, digital signatures**, and other cryptographic applications where a fixed-length output is required.
- It has been widely used in various **software applications**, including **web browsers, email clients, and operating systems**.

SHA-1 (Secure Hash Algorithm 1)

Limitation:

- **Vulnerabilities:** SHA-1 has been found to be vulnerable to **collision attacks**, which can lead to security breaches. Due to these vulnerabilities, SHA-1 **is no longer considered secure** for new applications.
- **Slow compared to newer algorithms:** SHA-1 is slower than newer hash functions such as SHA-2 and SHA-3.
- **Fixed-length output:** SHA-1 produces a fixed-length **output of 160 bits**, which may not be suitable for some applications that require longer hash values.
- **Not quantum-resistant:** SHA-1 is not resistant to attacks from **quantum computers**, which may become a concern in the future as quantum computing technology advances.
- Overall, while SHA-1 was once widely used and considered secure, its vulnerabilities have led to its deprecation and replacement with more secure hash functions. Newer algorithms like **SHA-2 and SHA-3 provide better** security and are recommended for use in modern cryptographic applications.

HMAC stands for Hash-based Message Authentication Code

- HMAC stands for Hash-based Message Authentication Code. It is a type of message authentication code that provides a way to verify the **integrity and authenticity** of a message.
- HMAC also stands for "Keyed-Hash Message Authentication Code". It is a type of message authentication code (MAC) that is used to verify the **integrity and authenticity of a message**. HMAC uses a **secret key** to generate a hash value from the message, which can be used to ensure that the **message has not been tampered** with during transmission.

Procedure for generating HMAC:

1. Choose a secret key.
2. Hash the key with the chosen hash function.
3. If the length of the hash output is greater than the block size of the hash function, hash the output again to obtain a key of the correct length.
4. XOR the key with the outer padding (a sequence of 0x5c bytes) and with the inner padding (a sequence of 0x36 bytes).
5. Hash the result of step 4 along with the message.
6. XOR the key with the outer padding and with the inner padding again.
7. Hash the result of step 6 along with the output of step 5.
8. The final result is the HMAC

HMAC stands for Hash-based Message Authentication Code

Applications of HMAC:

1. HMAC is commonly used in network security protocols like **SSL/TLS, IPsec, and SSH** to provide message integrity and authentication.
2. HMAC can be used to protect against attacks like message **tampering, replay attacks, and man-in-the-middle attacks**.
3. HMAC is also used in software applications like **password managers, digital signature systems, and two-factor authentication systems**.

Limitations of HMAC:

1. HMAC does not provide confidentiality, as the message can still be **read by anyone who intercepts it**.
2. HMAC is vulnerable to attacks like **brute force attacks**, where an attacker tries to guess the secret key by repeatedly generating HMACs until a match is found.
3. HMAC does not protect against attacks like **denial-of-service attacks**, where an attacker floods a system with a large number of requests to overwhelm it.

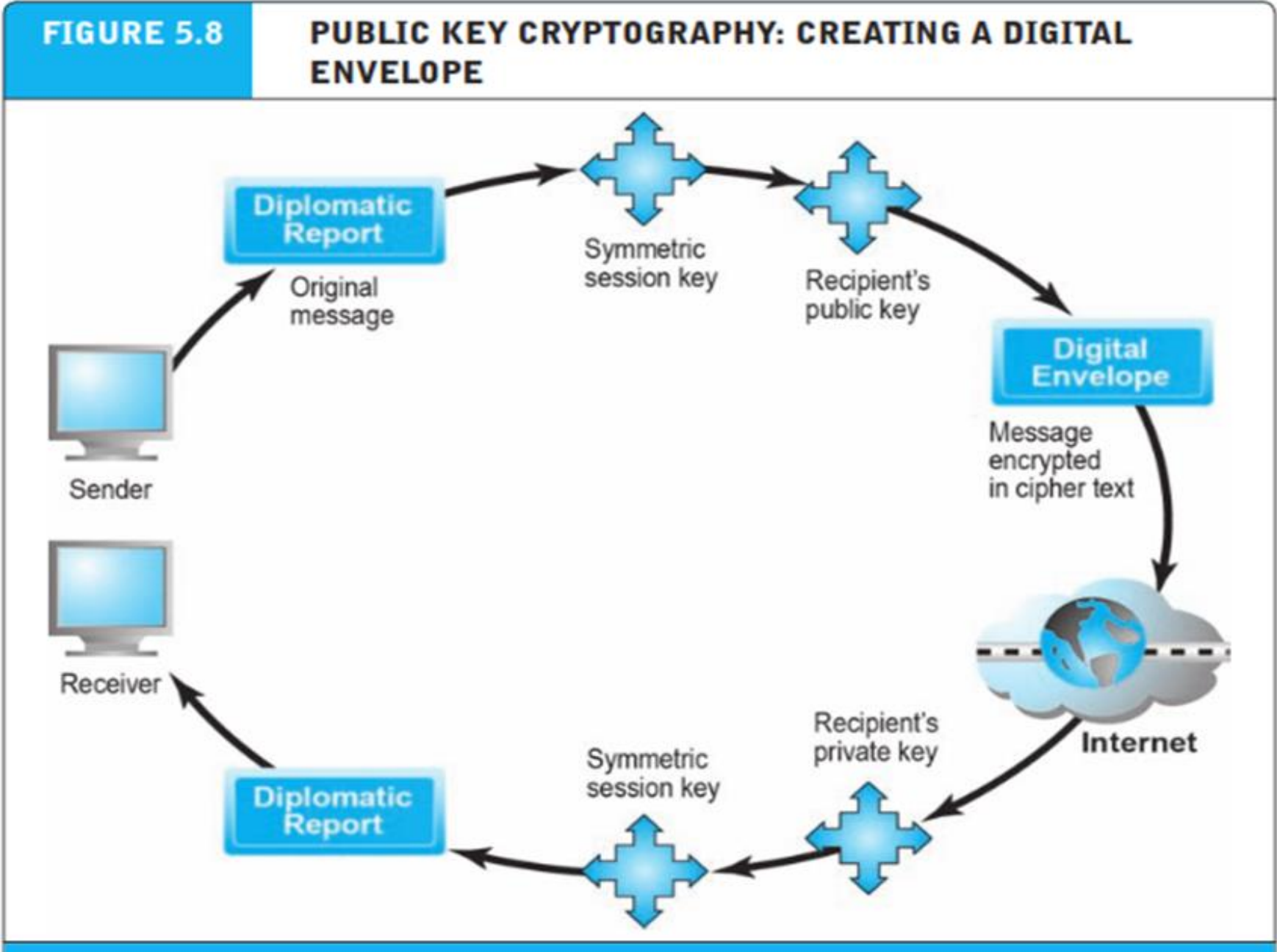
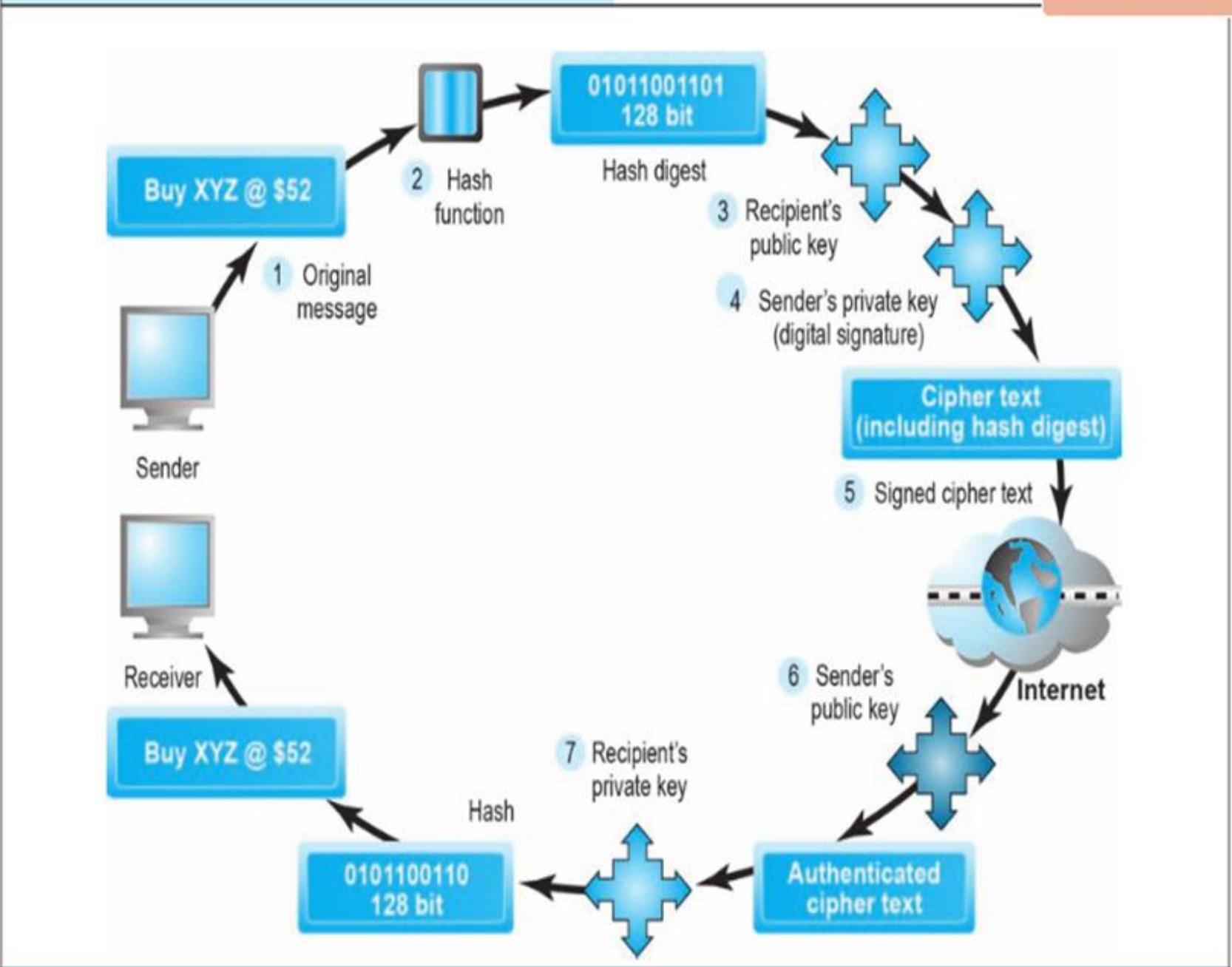
Digital signature

A digital signature is a **mathematical technique** used to ensure the **authenticity, integrity, and non-repudiation of electronic documents**, messages, or transactions.

Procedures:

- **Creation of digital signature:** The sender **creates a digital signature** by using a cryptographic algorithm to generate a unique digital code, which is based on the content of the message or document. The sender's **private key** is used to generate the **digital signature**.
- **Verification of digital signature:** The **recipient uses the sender's public key** to verify the digital signature. The recipient computes a new digital code based on the received message or document and compares it to the digital signature. **If the two codes** match, then the digital signature is valid, and the message or document is considered authentic.
- **Non-repudiation:** Since the **sender's private key** is used to generate the digital signature, the sender cannot deny that they signed the document or message. This property is known as non-repudiation.
- **Integrity:** Any change made to the original message or document will result in a different digital signature, making it impossible for the recipient to verify the signature.
- **Authenticity:** The digital signature proves the **identity of the sender**, as only the **sender's private key** can be used to generate the signature.

Digital signature



Digital signature

FIGURE 5.7 PUBLIC KEY CRYPTOGRAPHY WITH DIGITAL SIGNATURES	
STEP	DESCRIPTION
1. The sender creates an original message.	The message can be any digital file.
2. The sender applies a hash function, producing a 128-bit hash result.	Hash functions create a unique digest of the message based on the message contents.
3. The sender encrypts the message and hash result using recipient's public key.	This irreversible process creates a cipher text that can be read only by the recipient using his or her private key.
4. The sender encrypts the result, again using his or her private key.	The sender's private key is a digital signature. There is only one person who can create this digital mark.
5. The result of this double encryption is sent over the Internet.	The message traverses the Internet as a series of independent packets.
6. The receiver uses the sender's public key to authenticate the message.	Only one person can send this message, namely, the sender.
7. The receiver uses his or her private key to decrypt the hash function and the original message. The receiver checks to ensure the original message and the hash function results conform to one another.	The hash function is used here to check the original message. This ensures the message was not changed in transit.